torch.addbmm#

https://pytorch.org/docs/stable/generated/torch.addbmm.html

Description:

Performs a batch matrix-matrix product of matrices stored in batch1 and batch2, with a reduced add step (all matrix multiplications get accumulated along the first dimension). input is added to the final result. batch1 and batch2 must be 3-D tensors each containing the same number of matrices. If batch1 is a (b×n×m)(b \times n \times m)(b×n×m) tensor, batch2 is a (b×m×p)(b \times m \times p)(b×m×p) tensor, input must be broadcastable with a (n×p)(n \times p)(n×p) tensor and out will be a (n×p)(n \times p)(n×p) tensor. If beta is 0, then the content of input will be ignored, and nan and inf in it will not be propagated. For inputs of type FloatTensor or DoubleTensor, arguments beta and alpha must be real numbers, otherwise they should be integers.

Function Signature

```
torch.addbmm(input, batch1, batch2, *, beta=1, alpha=1,
out=None) → Tensor#

Parameters

Keyword Arguments
```

Parameters

Keyword

beta (Number, optional) – multiplier for input (β\betaβ) alpha (Number, optional) – multiplier for batch1 @ batch2 (α\alphaα) out (Tensor, optional) – the output tensor.

Code Examples

```
>>> M = torch.randn(3, 5)
>>> batch1 = torch.randn(10, 3, 4)
>>> batch2 = torch.randn(10, 4, 5)
>>> torch.addbmm(M, batch1, batch2)
tensor([[  6.6311,   0.0503,   6.9768, -12.0362,  -2.1653],
        [ -4.8185,  -1.4255,  -6.6760,   8.9453,   2.5743],
        [ -3.8202,   4.3691,   1.0943,  -1.1109,   5.4730]])
```

Detailed Documentation

Documentation

Performs a batch matrix-matrix product of matrices stored in batch1 and batch2, with a reduced add step (all matrix multiplications get accumulated along the first dimension). input is added to the final result.

batch1 and batch2 must be 3-D tensors each containing the same number of matrices.

If batch1 is a (b×n×m)(b \times n \times m)(b×n×m) tensor, batch2 is a (b×m×p)(b \times m \times p)(b×m×p) tensor, input must be broadcastable with a (n×p)(n \times p)(n×p) tensor and out will be a (n×p)(n \times p)(n×p) tensor.

If beta is 0, then the content of input will be ignored, and nan and inf in it will not be propagated.

For inputs of type FloatTensor or DoubleTensor, arguments beta and alpha must be

real numbers, otherwise they should be integers.

This operator supports TensorFloat32.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

input (Tensor) – matrix to be added

batch1 (Tensor) – the first batch of matrices to be multiplied

batch2 (Tensor) – the second batch of matrices to be multiplied

beta (Number, optional) – multiplier for input (β\betaβ)

alpha (Number, optional) – multiplier for batch1 @ batch2 (α\alphaα)

out (Tensor, optional) – the output tensor.
